

-  Operations Runbook — Multicloud Clearing Engine
 -  Table of Contents
 - 1. Quick Reference Card
 - Cluster Access Commands
 - Key Endpoints
 - Critical Thresholds
 - 2. Day-to-Day Operations
 - 2.1 Health Check Routine (Daily)
 - 2.2 ArgoCD Application Management
 - 2.3 Log Access
 - 3. Deployment Procedures
 - 3.1 Understanding the Two Pipeline Types
 - 3.2 Application Deployment (Standard via CI/CD)
 - 3.3 Application Deployment (Manual)
 - 3.4 Infrastructure Deployment (via CI/CD)
 - 3.5 Emergency Hotfix Deployment
 - 3.6 Terraform State Operations
 - 4. Application Container Operations
 - 4.1 Container Registry Management
 - 4.2 Helm Release Management
 - 4.3 Application Pod Operations
 - 5. Scaling Operations
 - 5.1 Application Scaling (HPA & Helm)
 - 5.2 EKS Node Group Scaling
 - 5.3 Aurora Read Replica Scaling
 - 5.4 AKS Node Pool Scaling
 - 6. Monitoring & Alerting
 - 6.1 Dashboard URLs
 - 6.2 Key Metrics to Watch
 - 7. Incident Response Playbooks
 - 7.1 PLAYBOOK: High API Latency (P99 > 2s)
 - 7.2 PLAYBOOK: VPN Tunnel Down
 - 7.3 PLAYBOOK: Database Failover
 - 7.4 PLAYBOOK: Application Pod CrashLoopBackOff
 - 8. Disaster Recovery Procedures
 - 8.1 AWS → Azure Failover (Full Region Failure)
 - 8.2 Failback: Azure → AWS Recovery
 - 9. Database Operations
 - 9.1 Aurora PostgreSQL

- [9.2 Azure SQL Hyperscale](#)
- [9.3 BigQuery Audit Queries](#)
- [10. Certificate & Secret Rotation](#)
 - [10.1 VPN Pre-Shared Keys](#)
 - [10.2 Database Credentials](#)
 - [10.3 KMS Key Rotation](#)
- [11. Compliance Audit Procedures](#)
 - [11.1 HIPAA Audit Checklist](#)
 - [11.2 SOX Audit Checklist](#)
 - [11.3 GDPR Audit Checklist](#)
- [12. VPN Troubleshooting](#)
 - [12.1 Diagnostic Commands](#)
 - [12.2 Common Issues](#)
- [13. Cost Management](#)
 - [13.1 Estimated Monthly Costs](#)
 - [13.2 Cost Optimization Tips](#)
- [14. Contact & Escalation](#)
 - [Escalation Matrix](#)



Operations Runbook — Multicloud Clearing Engine

Version: 1.0 | **Last Updated:** June 2026 | **Classification:** CONFIDENTIAL



Table of Contents

1. [Quick Reference Card](#)
2. [Day-to-Day Operations](#)
3. [Deployment Procedures](#)
4. [Application Container Operations](#)
5. [Scaling Operations](#)
6. [Monitoring & Alerting](#)
7. [Incident Response Playbooks](#)
8. [Disaster Recovery Procedures](#)
9. [Database Operations](#)
10. [Certificate & Secret Rotation](#)
11. [Compliance Audit Procedures](#)

- 12. [VPN Troubleshooting](#)
- 13. [Cost Management](#)
- 14. [Contact & Escalation](#)

1. Quick Reference Card

Cluster Access Commands

```
# — AWS EKS —
aws eks update-kubeconfig --name multicloud-clearing-engine-production-eks --
region us-east-1
kubectl config use-context arn:aws:eks:us-east-1:<ACCOUNT_ID>:cluster/multicloud-
clearing-engine-production-eks

# — Azure AKS —
az aks get-credentials --resource-group multicloud-clearing-engine-production-rg \
--name multicloud-clearing-engine-production-aks
kubectl config use-context multicloud-clearing-engine-production-aks

# — GCP GKE —
gcloud container clusters get-credentials multicloud-clearing-engine-production-
gke \
--region us-central1 --project enterprise-compliance-analytics
kubectl config use-context gke_enterprise-compliance-analytics_us-
central1_multicloud-clearing-engine-production-gke
```

Key Endpoints

Infrastructure Endpoints:

Service	Endpoint	Cloud
Primary API	<code>api.clearing-engine.example.com</code>	AWS (Route 53)
Standby API	Front Door endpoint	Azure
ArgoCD (AWS)	<code>argocd.internal.eks</code>	AWS EKS
ArgoCD (Azure)	<code>argocd.internal.aks</code>	Azure AKS

Service	Endpoint	Cloud
Aurora Writer	*.cluster-*.us-east-1.rds.amazonaws.com:5432	AWS
Aurora Reader	*.cluster-ro-*.us-east-1.rds.amazonaws.com:5432	AWS
Azure SQL	*.database.windows.net:1433	Azure
AlloyDB	Private IP (via VPN) :5432	GCP
Redis (AWS)	*.cache.amazonaws.com:6379	AWS
Redis (Azure)	*.redis.cache.windows.net:6380	Azure

Application Endpoints (inside clusters — namespace: `clearing-engine`):

Service	K8s Service	Port	Health	Metrics
transaction-ingestion	transaction-ingestion.clearing-engine.svc	8000	/health, /ready	/metrics
clearing-engine-core	clearing-engine-core.clearing-engine.svc	8000	/health, /ready	/metrics
audit-pipeline	audit-pipeline.clearing-engine.svc	8000	/health, /ready	/metrics
notification-service	notification-service.clearing-engine.svc	8000	/health, /ready	/metrics

Container Registries (Private Access Only):

Registry	URL	Access Method
AWS ECR	<ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com	VPC Interface Endpoints
Azure ACR	<ACR_NAME>.azurecr.io	Private Endpoint + DNS Zone

Critical Thresholds

Metric	Warning	Critical	Action
EKS Node CPU	> 70%	> 85%	Auto-scale triggers at 70%
Pod CPU (HPA)	> 70%	> 85%	HPA scales transaction-ingestion, clearing-engine
Aurora CPU	> 60%	> 80%	Scale reader instance
Redis Memory	> 75%	> 90%	Review eviction policy
VPN Tunnel Status	1 tunnel down	Both tunnels down	See §12 VPN Troubleshooting
Disk Usage	> 80%	> 90%	Expand EBS/Disk
API Latency (P99)	> 500ms	> 2000ms	Check DB connections
Pod Restarts	> 3 in 1hr	> 10 in 1hr	Check logs, OOM, image issues

2. Day-to-Day Operations

2.1 Health Check Routine (Daily)

Run this checklist every morning:

```
# =====
# Step 1: Verify all clusters are healthy
# =====
echo "=== AWS EKS ==="
kubectl --context=eks get nodes -o wide
kubectl --context=eks get pods -n clearing-engine --field-selector
status.phase!=Running

echo "=== Azure AKS ==="
kubectl --context=aks get nodes -o wide
kubectl --context=aks get pods -n clearing-engine --field-selector
status.phase!=Running

echo "=== GCP GKE ==="
kubectl --context=gke get nodes -o wide
kubectl --context=gke get pods -n clearing-engine --field-selector
status.phase!=Running

# =====
# Step 2: Verify APPLICATION pods are running
# =====
for ctx in eks aks; do
  echo "=== $ctx APPLICATION HEALTH ==="
```

```

kubectrl --context=$ctx get deploy -n clearing-engine
kubectrl --context=$ctx get hpa -n clearing-engine
for svc in transaction-ingestion clearing-engine-core audit-pipeline
notification-service; do
    echo "--- $svc ---"
    kubectrl --context=$ctx exec deploy/$svc -n clearing-engine -- \
        python -c "import httpx; r=httpx.get('http://localhost:8000/ready');
print(r.json())" 2>/dev/null || echo "SKIP"
done
done

# GKE runs audit-pipeline only
echo "=== GKE AUDIT PIPELINE ==="
kubectrl --context=gke exec deploy/audit-pipeline -n clearing-engine -- \
    python -c "import httpx; r=httpx.get('http://localhost:8000/ready');
print(r.json())" 2>/dev/null || echo "SKIP"

# =====
# Step 3: Verify Istio mesh health
# =====
kubectrl --context=eks -n istio-system get pods
istioctl --context=eks proxy-status

# =====
# Step 4: Check ArgoCD sync status
# =====
argocd app list --server argocd.internal.eks
# All apps should show "Synced" and "Healthy"

# =====
# Step 5: Verify VPN tunnel status
# =====
aws ec2 describe-vpn-connections --query 'VpnConnections[*].VgwTelemetry[*].
[OutsideIpAddress,Status]'

# =====
# Step 6: Check database connectivity
# =====
psql -h <aurora-writer-endpoint> -U admin -d clearingdb -c "SELECT 1;"

# =====
# Step 7: Check container registries
# =====
# ECR: Verify image scan results (no CRITICAL CVEs)
for svc in transaction-ingestion clearing-engine audit-pipeline notification-
service; do
    aws ecr describe-image-scan-findings \
        --repository-name multicloud-clearing-engine/$svc \
        --image-id imageTag=latest \
        --query 'imageScanFindings.findingSeverityCounts' 2>/dev/null || echo "$svc:
no scan"
done

# Step 8: Review overnight alerts
# Check CloudWatch, Azure Monitor, and GCP Cloud Monitoring dashboards

```

2.2 ArgoCD Application Management

```
# List all applications
argocd app list

# Check sync status
argocd app get clearing-engine-aws
argocd app get clearing-engine-azure
argocd app get clearing-engine-gcp

# Force sync (if auto-sync is paused)
argocd app sync clearing-engine-aws

# View application diff (what will change)
argocd app diff clearing-engine-aws

# Rollback to previous version
argocd app history clearing-engine-aws
argocd app rollback clearing-engine-aws <REVISION_ID>
```

2.3 Log Access

```
# =====
# APPLICATION LOGS (per-service)
# =====

# Live logs from a specific service (EKS)
kubectl --context=eks logs -f deploy/transaction-ingestion -n clearing-engine
kubectl --context=eks logs -f deploy/clearing-engine-core -n clearing-engine
kubectl --context=eks logs -f deploy/audit-pipeline -n clearing-engine
kubectl --context=eks logs -f deploy/notification-service -n clearing-engine

# Last 100 lines from all pods of a service
kubectl --context=eks logs deploy/transaction-ingestion -n clearing-engine --
tail=100 --all-containers

# Logs from a crashed/restarting pod (previous container)
kubectl --context=eks logs <pod-name> -n clearing-engine --previous

# =====
# INFRASTRUCTURE LOGS
# =====

# AWS CloudWatch Logs (cluster-level)
aws logs tail /aws/eks/multicloud-clearing-engine-production-eks/cluster --follow
aws logs tail /aws/cloudtrail/multicloud-clearing-engine-production --follow

# Azure Log Analytics (container logs)
az monitor log-analytics query \
  --workspace <WORKSPACE_ID> \
  --analytics-query "ContainerLog | where TimeGenerated > ago(1h) | limit 50"

# Azure – filter by specific application service
az monitor log-analytics query \
  --workspace <WORKSPACE_ID> \
  --analytics-query "ContainerLog | where ContainerName == 'transaction-
```

```

ingestion' | where TimeGenerated > ago(1h) | limit 50"

# GCP Cloud Logging
gcloud logging read "resource.type=k8s_container AND
resource.labels.cluster_name=multicloud-clearing-engine-production-gke" \
  --limit 50 --format json

# =====
# AUDIT & COMPLIANCE LOGS
# =====
bq query --use_legacy_sql=false \
  'SELECT * FROM `enterprise-compliance-
analytics.compliance_audit_logs.transaction_audit_trail`
  WHERE event_timestamp > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 24 HOUR)
  ORDER BY event_timestamp DESC LIMIT 100'

```

3. Deployment Procedures

3.1 Understanding the Two Pipeline Types

This project has **two separate CI/CD pipeline sets**:

Pipeline Type	Trigger	Files	What It Deploys
Infrastructure	Changes to terraform/	aws-deploy.yml , azure-deploy.yml , gcp-deploy.yml	VPCs, clusters, databases, registries, VPN
Application	Changes to src/ or helm/	ci-app.yml → cd-app.yml	Container images + Helm releases to clusters

3.2 Application Deployment (Standard via CI/CD)

When a developer pushes code changes to **src/**:

Step 1: **ci-app.yml** triggers automatically

For EACH service (parallel matrix: 4 services):

- Install Python 3.12 + dependencies
- Lint with ruff
- Run pytest unit tests
- Build Docker image (multi-stage, non-root)
- Trivy scan for CRITICAL/HIGH CVEs (fails on finding)

- f. TruffleHog scan for leaked secrets
 - g. Push multi-arch image to BOTH:
 - └─ AWS ECR (via OIDC → VPC Interface Endpoints)
 - └─ Azure ACR (via OIDC → Private Endpoint)
 - h. Generate SBOM (Software Bill of Materials)
- Image tags: sha-<commit>, latest

Step 2: cd-app.yml triggers after ci-app.yml succeeds on main

- a. Deploy to EKS (Primary Active)
 - OIDC auth → aws eks update-kubeconfig
 - helm upgrade --install --atomic
 - f values.yaml -f values-aws.yaml
 - kubectl rollout status (waits for pods)
 - Smoke test: curl /health on each service
- b. Deploy to AKS (Hot Standby) – after EKS succeeds
 - OIDC auth → az aks get-credentials
 - helm upgrade --install --atomic
 - f values.yaml -f values-azure.yaml
 - kubectl rollout status
- c. Deploy to GKE (Audit Only) – after EKS succeeds
 - OIDC auth → gcloud get-gke-credentials
 - helm upgrade --install --atomic
 - f values.yaml -f values-gcp.yaml
 - Only audit-pipeline is deployed
- d. Deployment Summary Report

3.3 Application Deployment (Manual)

```
# — Step 1: Build Docker images locally —
for svc in transaction-ingestion clearing-engine audit-pipeline notification-
service; do
  docker build -t $svc:latest src/$svc/
  echo "✓ Built $svc"
done

# — Step 2: Push to AWS ECR —
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.us-east-
1.amazonaws.com

for svc in transaction-ingestion clearing-engine audit-pipeline notification-
service; do
  docker tag $svc:latest <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/multicloud-
clearing-engine/$svc:latest
  docker push <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/multicloud-clearing-
engine/$svc:latest
  echo "✓ Pushed $svc to ECR"
done
```

```
# — Step 3: Push to Azure ACR —
az acr login --name <ACR_NAME>
for svc in transaction-ingestion clearing-engine audit-pipeline notification-
service; do
    docker tag $svc:latest <ACR_NAME>.azurecr.io/clearing-engine/$svc:latest
    docker push <ACR_NAME>.azurecr.io/clearing-engine/$svc:latest
    echo "✓ Pushed $svc to ACR"
done

# — Step 4: Deploy to EKS via Helm —
aws eks update-kubeconfig --name <EKS_CLUSTER_NAME> --region us-east-1
helm upgrade --install clearing-engine helm/clearing-engine/ \
    --namespace clearing-engine --create-namespace \
    -f helm/clearing-engine/values.yaml \
    -f helm/clearing-engine/values-aws.yaml \
    --set global.imageRegistry=<ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com \
    --set global.imageTag=latest \
    --wait --atomic --timeout 10m

# — Step 5: Deploy to AKS via Helm —
az aks get-credentials --resource-group <RG_NAME> --name <AKS_CLUSTER_NAME>
helm upgrade --install clearing-engine helm/clearing-engine/ \
    --namespace clearing-engine --create-namespace \
    -f helm/clearing-engine/values.yaml \
    -f helm/clearing-engine/values-azure.yaml \
    --set global.imageRegistry=<ACR_NAME>.azurecr.io \
    --set global.imageTag=latest \
    --wait --atomic --timeout 10m

# — Step 6: Verify deployments —
kubectl get pods -n clearing-engine -o wide
kubectl get svc -n clearing-engine
kubectl get hpa -n clearing-engine
```

3.4 Infrastructure Deployment (via CI/CD)

1. Create a feature branch and make changes under **terraform/**
2. Open a Pull Request to **main**
3. CI pipeline runs: format check → validate → checkov scan → plan
4. Review the **terraform plan** output in the PR
5. Merge to **main** → Apply job runs with environment approval gate
6. ArgoCD auto-syncs Kubernetes manifests

3.5 Emergency Hotfix Deployment

```
# — For Terraform changes —
cd terraform/environments/production
terraform init
terraform plan -target=module.<affected_module> -out=hotfix.plan
```

```
terraform apply hotfix.plan
```

```
# — For Application changes (bypass CI) —
```

```
# Build, push, and deploy a single service:
```

```
SVC=transaction-ingestion
```

```
docker build -t $SVC:hotfix src/$SVC/
```

```
docker tag $SVC:hotfix <ECR_REGISTRY>/multicloud-clearing-engine/$SVC:hotfix
```

```
docker push <ECR_REGISTRY>/multicloud-clearing-engine/$SVC:hotfix
```

```
kubectl --context=eks set image deploy/$SVC $SVC=<ECR_REGISTRY>/multicloud-clearing-engine/$SVC:hotfix -n clearing-engine
```

```
kubectl --context=eks rollout status deploy/$SVC -n clearing-engine
```

```
# — Rollback to previous version —
```

```
kubectl --context=eks rollout undo deploy/$SVC -n clearing-engine
```

```
# OR via Helm:
```

```
helm rollback clearing-engine <REVISION> --namespace clearing-engine
```

3.6 Terraform State Operations

```
# View current state
```

```
terraform state list
```

```
# Import an existing resource
```

```
terraform import module.aws_infra.aws_vpc.main vpc-xxxxx
```

```
# Remove a resource from state (without destroying)
```

```
terraform state rm module.aws_infra.aws_vpc.main
```

```
# Move a resource in state (rename)
```

```
terraform state mv module.old.resource module.new.resource
```

```
# Force unlock state (if lock is stale)
```

```
terraform force-unlock <LOCK_ID>
```

4. Application Container Operations

4.1 Container Registry Management

```
# =====
```

```
# AWS ECR – List images and scan results
```

```
# =====
```

```
for svc in transaction-ingestion clearing-engine audit-pipeline notification-service; do
```

```
    echo "=== $svc ==="
```

```
    aws ecr list-images --repository-name multicloud-clearing-engine/$svc \
```

```
        --query 'imageIds[*].imageTag' --output text
```

```
    echo ""
```

done

```
# Check vulnerability scan results for a specific image
aws ecr describe-image-scan-findings \
  --repository-name multicloud-clearing-engine/transaction-ingestion \
  --image-id imageTag=latest

# =====
# Azure ACR – List images and scan results
# =====
az acr repository list --name <ACR_NAME> --output table
az acr repository show-tags --name <ACR_NAME> \
  --repository clearing-engine/transaction-ingestion --output table
```

4.2 Helm Release Management

```
# — View current release status —
helm list -n clearing-engine
helm status clearing-engine -n clearing-engine

# — View release history (for rollback) —
helm history clearing-engine -n clearing-engine

# — View what's deployed vs what's in chart —
helm diff upgrade clearing-engine helm/clearing-engine/ \
  -f helm/clearing-engine/values.yaml \
  -f helm/clearing-engine/values-aws.yaml \
  -n clearing-engine

# — Rollback to a previous revision —
helm rollback clearing-engine <REVISION> -n clearing-engine

# — View computed values (what was actually applied) —
helm get values clearing-engine -n clearing-engine --all
```

4.3 Application Pod Operations

```
# — Check pod status across all clusters —
for ctx in eks aks gke; do
  echo "=== $ctx ==="
  kubectl --context=$ctx get pods -n clearing-engine -o wide
done

# — Restart a single service (rolling restart) —
kubectl --context=eks rollout restart deploy/transaction-ingestion -n clearing-engine

# — Watch rollout progress —
kubectl --context=eks rollout status deploy/transaction-ingestion -n clearing-engine --watch
```

```
# — Exec into a running pod (debugging) —
kubectl --context=eks exec -it deploy/transaction-ingestion -n clearing-engine --
/bin/sh

# — Port-forward to test locally —
kubectl --context=eks port-forward svc/transaction-ingestion 8000:8000 -n
clearing-engine
# Then: curl http://localhost:8000/health

# — View NetworkPolicies (pod-to-pod rules) —
kubectl --context=eks get networkpolicies -n clearing-engine -o wide
```

5. Scaling Operations

5.1 Application Scaling (HPA & Helm)

```
# — View current HPA status —
kubectl --context=eks get hpa -n clearing-engine
# Expected: transaction-ingestion (3-10), clearing-engine-core (3-15)

# — Manual scale (temporary, HPA will override) —
kubectl --context=eks scale deploy/transaction-ingestion --replicas=8 -n
clearing-engine

# — Permanent scaling via Helm values —
# Edit values-aws.yaml:
#   transaction-ingestion:
#     autoscaling:
#       minReplicas: 5
#       maxReplicas: 20
# Then deploy:
helm upgrade clearing-engine helm/clearing-engine/ \
  --namespace clearing-engine \
  -f helm/clearing-engine/values.yaml \
  -f helm/clearing-engine/values-aws.yaml \
  --wait --atomic

# — View Pod Disruption Budgets —
kubectl --context=eks get pdb -n clearing-engine
```

5.2 EKS Node Group Scaling

```
# View current capacity
aws eks describe-nodegroup \
  --cluster-name multicloud-clearing-engine-production-eks \
  --nodegroup-name multicloud-clearing-engine-production-main-ng \
```

```
--query 'nodegroup.scalingConfig'

# Manual scale (temporary)
aws eks update-nodegroup-config \
  --cluster-name multicloud-clearing-engine-production-eks \
  --nodegroup-name multicloud-clearing-engine-production-main-ng \
  --scaling-config minSize=5,maxSize=15,desiredSize=5

# Permanent change: Update terraform.tfvars
# aws_eks_node_min = 5
# aws_eks_node_max = 15
# Then run: terraform apply
```

5.3 Aurora Read Replica Scaling

```
# Add a read replica (via Terraform)
# In aurora.tf, add another aws_rds_cluster_instance resource
# terraform apply

# Monitor replication lag
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name AuroraReplicaLag \
  --dimensions Name=DBInstanceIdentifier,Value=multicloud-clearing-engine-
production-aurora-reader \
  --start-time $(date -u -v-1H +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --period 300 \
  --statistics Average
```

5.4 AKS Node Pool Scaling

```
# Scale user node pool
az aks nodepool scale \
  --resource-group multicloud-clearing-engine-production-rg \
  --cluster-name multicloud-clearing-engine-production-aks \
  --name user \
  --node-count 5
```

6. Monitoring & Alerting

6.1 Dashboard URLs

Dashboard	URL	Purpose
AWS CloudWatch	console.aws.amazon.com/cloudwatch	EKS, Aurora, ElastiCache metrics
Azure Monitor	portal.azure.com/#blade/Microsoft_Azure_Monitoring	AKS, SQL, Redis metrics
GCP Monitoring	console.cloud.google.com/monitoring	GKE, AlloyDB metrics
Grafana (if deployed)	grafana.internal.eks	Unified dashboards
Kiali (Istio)	kiali.internal.eks	Service mesh visualization

6.2 Key Metrics to Watch

Application Metrics (Prometheus — scraped from [/metrics](#)):

- [transactions_received_total](#) — total transactions ingested (by type, status)
- [transaction_processing_seconds](#) — ingestion latency histogram
- [clearing_operations_total](#) — total clearing operations
- [clearing_latency_seconds](#) — clearing processing time
- [audit_events_total](#) — audit events streamed (by type, cloud source)
- [notifications_sent_total](#) — notifications sent (by channel, status)

Transaction Processing:

- Transaction throughput (TPS)
- Transaction latency (P50, P95, P99)
- Error rate (4xx, 5xx)
- Queue depth (if using SQS/ASB)

Infrastructure:

- Node CPU/Memory utilization
- Pod restart count

- Network throughput (VPN tunnels)
 - Database connections (active/idle)
 - Cache hit ratio
-

7. Incident Response Playbooks

7.1 PLAYBOOK: High API Latency (P99 > 2s)

SEVERITY: P2 – High
RESPONSE TIME: 15 minutes

DIAGNOSIS:

1. Check which cloud is affected:
 - AWS: Check CloudFront → ALB → EKS pod latency breakdown
 - Azure: Check Front Door → Application Gateway → AKS pod latency
2. Check database:
 - `aws cloudwatch get-metric-statistics` (Aurora CPUUtilization)
 - Check for long-running queries: `SELECT * FROM pg_stat_activity WHERE state != 'idle';`
3. Check cache:
 - Redis cache hit ratio (should be > 90%)
 - ElastiCache: `aws cloudwatch get-metric-statistics --metric-name CacheHitRate`
4. Check Istio:
 - `istioctl proxy-status` (look for STALE or NOT CONNECTED)

REMEDIATION:

- a. Database slow: Kill long queries, add read replica
- b. Cache cold: Pre-warm cache, increase Redis memory
- c. Pod overload: Scale horizontally (`kubectl scale deployment`)
- d. Network: Check VPN tunnel status, re-establish if needed

7.2 PLAYBOOK: VPN Tunnel Down

SEVERITY: P1 – Critical (if both tunnels to same cloud are down)
RESPONSE TIME: 5 minutes

DIAGNOSIS:

1. Identify which tunnel is down:
 - `aws ec2 describe-vpn-connections --query 'VpnConnections[*].[VpnConnectionId,VgwTelemetry[*].Status]'`
2. Check Azure side:
 - `az network vpn-connection show --name <connection-name> \`
`--resource-group multicloud-clearing-engine-production-rg`

3. Check GCP side:
`gcloud compute vpn-tunnels describe <tunnel-name> --region us-central1`

REMEDIATION:

- a. Single tunnel down (redundant active): Monitor, will auto-recover
- b. Both tunnels down to same cloud:
 - Reset the VPN connection:
`aws ec2 modify-vpn-connection --vpn-connection-id <id>`
 - Verify PSK matches on both ends
 - Check security group / firewall rules for UDP 500, 4500
- c. If persistent: Re-create tunnel via Terraform
`terraform apply -target=module.<cloud>_infra.aws_vpn_connection.<peer>`

7.3 PLAYBOOK: Database Failover

7.4 PLAYBOOK: Application Pod CrashLoopBackOff

SEVERITY: P2 – High
RESPONSE TIME: 15 minutes

DIAGNOSIS:

1. Identify which service and cluster:
`kubectl get pods -n clearing-engine --field-selector status.phase!=Running`
2. Check pod events:
`kubectl describe pod <pod-name> -n clearing-engine`
3. Check previous container logs:
`kubectl logs <pod-name> -n clearing-engine --previous`
4. Common causes:
 - a. OOMKilled → check `resources.limits.memory` in Helm values
 - b. Image pull failure → verify ECR/ACR private endpoint connectivity
 - c. Readiness probe failure → check /ready endpoint, DB, Redis connectivity
 - d. Config error → check env vars via: `kubectl exec <pod> -- env`

REMEDIATION:

- a. OOM: Increase memory limits in `values-aws.yaml` / `values-azure.yaml`
- b. Image pull: Verify VPC endpoints (ECR) or Private Endpoint (ACR)
- c. DB issue: Check Aurora/Azure SQL connectivity from within the pod
- d. Rollback: `helm rollback clearing-engine <previous-revision> -n clearing-engine`

SEVERITY: P1 – Critical
RESPONSE TIME: Immediate

AWS AURORA FAILOVER:

1. Aurora handles automatic failover (reader → writer)

2. Verify: `aws rds describe-db-clusters --db-cluster-identifier <cluster-id>`
3. Application connection strings auto-update (cluster endpoint)
4. Monitor replication lag post-failover

AZURE SQL FAILOVER:

1. `az sql db failover --resource-group <rg> --server <server> --name <db>`
2. Verify new primary is healthy
3. Check application connectivity

FULL AWS → AZURE FAILOVER:

See §7 Disaster Recovery Procedures

8. Disaster Recovery Procedures

8.1 AWS → Azure Failover (Full Region Failure)

RTO Target: 15 minutes | RPO Target: < 1 minute

STEP 1: Confirm AWS is down (2 min)

- Check AWS Health Dashboard
- Verify Route 53 health checks are failing
- Confirm CloudFront returns 5xx

STEP 2: Verify Azure standby is ready (2 min)

- `kubectl --context=aks get nodes` (all Ready?)
- `kubectl --context=aks get pods -n clearing-engine` (all Running?)
- Test Azure SQL connectivity

STEP 3: Update DNS to point to Azure (5 min)

- Route 53 should auto-failover if health checks are configured
- Manual override if needed:

```
aws route53 change-resource-record-sets \  
  --hosted-zone-id <ZONE_ID> \  
  --change-batch '{"Changes":[{"Action":"UPSERT","ResourceRecordSet":  
{"Name":"api.clearing-  
engine.example.com","Type":"CNAME","TTL":60,"ResourceRecords":[{"Value":  
<AZURE_FRONTDOOR_ENDPOINT>}]}]}'
```

STEP 4: Scale Azure AKS + application (3 min)

- `az aks nodepool scale --resource-group <rg> --cluster-name <cluster> \
 --name user --node-count 10`
- Scale application replicas to production levels:
`helm upgrade clearing-engine helm/clearing-engine/ \
 -n clearing-engine \
 -f values.yaml -f values-azure.yaml \
 --set transaction-ingestion.replicaCount=3 \
 --set clearing-engine-core.replicaCount=3 \
 --wait --atomic`

STEP 5: Verify traffic is flowing (3 min)

- Monitor Azure Front Door access logs

- Verify transaction processing in Azure SQL
- Check BigQuery audit trail shows Azure-sourced events

STEP 6: Notify stakeholders

- Send incident notification
- Update status page

8.2 Failback: Azure → AWS Recovery

STEP 1: Confirm AWS services are restored

- Check all EKS nodes are Ready
- Verify Aurora writer is healthy
- Confirm VPN tunnels are re-established

STEP 2: Sync data changes

- Any transactions processed by Azure SQL during failover must be reconciled with Aurora
- Run data synchronization scripts

STEP 3: Gradual traffic shift

- Route 53 weighted routing: 10% AWS / 90% Azure
- Monitor for errors
- Increase: 50/50, 90/10, 100/0

STEP 4: Scale down Azure standby

- Return Azure node pools to standby sizing

9. Database Operations

9.1 Aurora PostgreSQL

```
# Connect to writer
psql -h <writer-endpoint> -p 5432 -U admin -d clearingdb

# Check replication status
SELECT * FROM aurora_replica_status();

# Kill long-running queries
SELECT pg_terminate_backend(pid) FROM pg_stat_activity
WHERE duration > interval '5 minutes' AND state != 'idle';

# Create a manual snapshot
aws rds create-db-cluster-snapshot \
  --db-cluster-identifier multicloud-clearing-engine-production-aurora \
  --db-cluster-snapshot-identifier manual-snap-$(date +%Y%m%d)

# Point-in-time recovery
```

```
aws rds restore-db-cluster-to-point-in-time \  
  --source-db-cluster-identifier multicloud-clearing-engine-production-aurora \  
  --db-cluster-identifier restored-cluster \  
  --restore-to-time "2026-06-03T15:00:00Z"
```

9.2 Azure SQL Hyperscale

```
# Connect via sqlcmd  
sqlcmd -S <server>.database.windows.net -d clearingdb -U sqladmin -P <password>  
  
# Check active sessions  
SELECT * FROM sys.dm_exec_sessions WHERE is_user_process = 1;  
  
# Long-term backup retention check  
az sql db ltr-backup list --location eastus --server <server> --database  
clearingdb
```

9.3 BigQuery Audit Queries

```
-- Recent transactions (last 24 hours)  
SELECT event_id, event_timestamp, cloud_provider, transaction_type,  
       amount_cents, currency, status  
FROM `compliance_audit_logs.transaction_audit_trail`  
WHERE event_timestamp > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 24 HOUR)  
ORDER BY event_timestamp DESC  
LIMIT 1000;  
  
-- PII access log (who accessed what?)  
SELECT access_timestamp, accessor_id, accessor_role, resource_type, action  
FROM `compliance_audit_logs.pii_access_log`  
WHERE access_timestamp > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)  
ORDER BY access_timestamp DESC;  
  
-- Configuration drift events  
SELECT detected_at, cloud_provider, resource_type, severity, remediated  
FROM `compliance_audit_logs.config_drift_events`  
WHERE remediated = FALSE  
ORDER BY severity DESC, detected_at DESC;
```

10. Certificate & Secret Rotation

10.1 VPN Pre-Shared Keys

```
# Generate new PSK
NEW_PSK=$(openssl rand -base64 48)

# Update in all three clouds simultaneously:
# 1. AWS: terraform apply -var="vpn_shared_secret_aws_azure=$NEW_PSK"
# 2. Azure: Update Local Network Gateway shared key
# 3. GCP: Update VPN tunnel shared secret
# ⚠️ Brief connectivity disruption expected during rotation
```

10.2 Database Credentials

```
# Aurora: Rotate via Secrets Manager
aws secretsmanager rotate-secret --secret-id multicloud-clearing-engine-db-
password

# Azure SQL: Rotate via Key Vault
az keyvault secret set --vault-name <vault> --name sql-admin-password --value
<new-password>
az sql server update --resource-group <rg> --name <server> --admin-password <new-
password>
```

10.3 KMS Key Rotation

All KMS keys are configured with automatic rotation:

- **AWS KMS:** Annual automatic rotation (enabled in Terraform)
- **Azure Key Vault:** 90-day rotation policy
- **GCP Cloud KMS:** 90-day rotation period

Manual rotation is not typically required.

11. Compliance Audit Procedures

11.1 HIPAA Audit Checklist

#	Check	Command/Tool	Frequency
1	All storage encrypted at rest	AWS Config rule, Azure Policy	Daily (automated)
2	All traffic encrypted in transit	Istio mTLS STRICT, VPN status	Daily (automated)

#	Check	Command/Tool	Frequency
3	Access logs complete	CloudTrail, Azure Activity Log	Weekly review
4	Database audit logs	pgAudit, SQL Audit	Weekly review
5	PHI access justified	BigQuery pii_access_log	Monthly review
6	Backup verification	Test restore procedure	Quarterly
7	Incident response tested	Tabletop exercise	Annually

11.2 SOX Audit Checklist

#	Check	Command/Tool	Frequency
1	Separation of duties verified	Azure AD group membership audit	Monthly
2	All infrastructure changes logged	CloudTrail + Config	Continuous
3	Configuration drift detected	BigQuery config_drift_events	Weekly
4	Financial transactions immutable	BigQuery transaction_audit_trail	Daily
5	7-year log retention verified	S3 lifecycle, BQ retention	Quarterly

11.3 GDPR Audit Checklist

#	Check	Command/Tool	Frequency
1	EU data stays in EU	BigQuery EU dataset location	Monthly
2	PII tokenization active	Application audit	Weekly
3	Egress controls enforced	Istio Sidecar REGISTRY_ONLY	Daily
4	Right to erasure process	Manual verification	On-request
5	Data processing records	BigQuery pii_access_log	Monthly

12. VPN Troubleshooting

12.1 Diagnostic Commands

```
# — AWS Side —
aws ec2 describe-vpn-connections \
  --filters "Name=tag:Name,Values=*clearing-engine*" \
  --query 'VpnConnections[*]'.
{ID:VpnConnectionId,Status:State,Telemetry:VgwTelemetry[*].
{IP:OutsideIpAddress,Status:Status,StatusMsg:StatusMessage}}'

# — Azure Side —
az network vpn-connection list \
  --resource-group multicloud-clearing-engine-production-rg \
  --query '[]'.
{Name:name,Status:connectionStatus,Egress:egressBytesTransferred,Ingress:ingressB
ytesTransferred}'

# — GCP Side —
gcloud compute vpn-tunnels list \
  --filter="name~clearing-engine" \
  --format="table(name,status,peerIp,detailedStatus)"
```

12.2 Common Issues

Symptom	Likely Cause	Fix
Tunnel UP but no traffic	BGP not established	Check ASN configuration (65000/65001/65002)
Tunnel DOWN, IKE timeout	PSK mismatch	Verify shared secret on both ends
Intermittent drops	MTU issues	Set MTU to 1400 on tunnel interfaces
High latency over VPN	Packet loss	Check internet connection between clouds

13. Cost Management

13.1 Estimated Monthly Costs

Component	AWS	Azure	GCP	Total
Compute (K8s)	~\$2,500	~\$2,000	~\$1,200	~\$5,700
Database	~\$3,000	~\$2,500	~\$1,500	~\$7,000
Caching	~\$1,200	~\$800	—	~\$2,000
CDN/WAF	~\$500	~\$400	—	~\$900

Component	AWS	Azure	GCP	Total
VPN	~\$200	~\$400	~\$200	~\$800
Monitoring	~\$300	~\$200	~\$100	~\$600
Subtotal	~\$7,700	~\$6,300	~\$3,000	~\$17,000

13.2 Cost Optimization Tips

- Use **Savings Plans** (AWS) / **Reserved Instances** (Azure/GCP) for 1-year commit
 - Scale down **Azure standby** node pools during off-peak hours
 - Use **Aurora Serverless v2** for dev/staging environments
 - Archive **BigQuery** audit data older than 90 days to cold storage
 - Review **CloudFront** cache hit ratio — higher = lower origin costs
-

14. Contact & Escalation

Level	Team	Contact	Response SLA
L1	NOC / On-Call	PagerDuty rotation	5 min
L2	Platform Engineering	Slack: #platform-eng	15 min
L3	Cloud Architects	Direct page	30 min
L4	VP Engineering	Phone escalation	1 hour

Escalation Matrix

P1 (Critical – Service Down):	L1 → L2 (5 min) → L3 (15 min) → L4 (30 min)
P2 (High – Degraded Service):	L1 → L2 (15 min) → L3 (1 hour)
P3 (Medium – Non-Critical):	L1 → L2 (1 hour)
P4 (Low – Informational):	L1 (next business day)

This runbook is a living document. Update after every incident or significant infrastructure change.